

BABU BANARSI DAS UNIVERSITY

LUCKNOW

SESSION: 2025-26



SCHOOL OF COMPUTER APPLICATION

Case Study on

Deep Work and Digital Fatigue Analytics

Submitted by

Omi Kumar – 1240258308

Prateesh Srivastava – 1240258326

Richa Kumari- 1240258362

Mansi Singh – 1240258251

Submitted to

Mr. Vikas

Project Overview & Description

- This project investigates the modern productivity crisis among students by analyzing the correlation between **digital habits (specifically short-form content consumption)** and **cognitive performance**.
- While traditional metrics often confuse total desk hours with productivity, this analysis isolates "**Deep Work Capacity**" against the variables of **sleep duration** and "**Doomscrolling**."
- By processing **5,000 synthetic student records**, the project aims to definitively categorize **how high-stress environments, coupled with excessive social media consumption, result in severe mental fatigue, thereby destroying actual academic efficiency.**

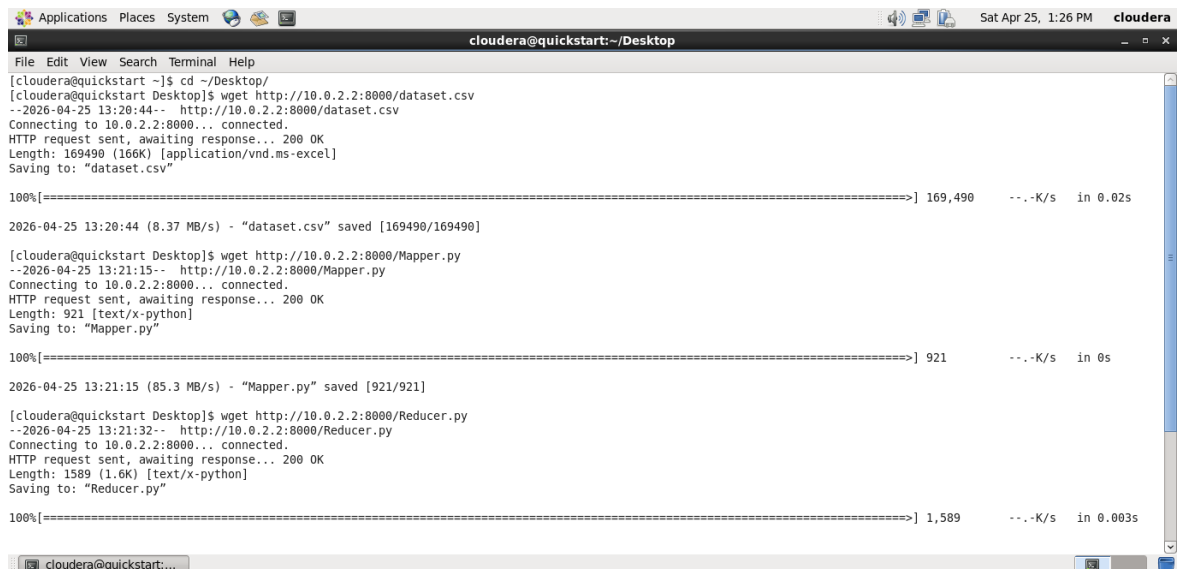
1. System Architecture & Data Flow

- **Phase 1 (Storage):** The raw CSV dataset is ingested into the Hadoop Distributed File System (HDFS), ensuring fault-tolerant and distributed storage across the cluster.
- **Phase 2 (Processing):** A Python-based MapReduce engine, executed via Hadoop Streaming, processes the raw data. The Mapper isolates relevant high-stress data points, and the Reducer aggregates these points to calculate the mathematical averages.
- **Phase 3 (Analytics):** The processed, aggregated data is exported from HDFS and loaded into an Apache Hive data warehouse, where analytical queries extract the final business insights.

- **Data Flow Diagram:** Raw Data -> HDFS (/input) -> MapReduce Transformation -> HDFS (/output) -> Hive Table -> Analytical Insights.

2. Methodology & Execution Workflow

- **Step 1: Data Staging in HDFS:** The environment was initialized, and the 5,000-row synthetic dataset was successfully transferred from the local Linux file system into the HDFS /input directory.



```
Applications Places System cloudera@quickstart:~/Desktop
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ cd ~/Desktop/
[cloudera@quickstart Desktop]$ wget http://10.0.2.2:8000/dataset.csv
--2026-04-25 13:20:44-- http://10.0.2.2:8000/dataset.csv
Connecting to 10.0.2.2:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 169490 (166K) [application/vnd.ms-excel]
Saving to: "dataset.csv"

100%[=====] 169,490  --.-K/s  in 0.02s

2026-04-25 13:20:44 (8.37 MB/s) - "dataset.csv" saved [169490/169490]

[cloudera@quickstart Desktop]$ wget http://10.0.2.2:8000/Reducer.py
--2026-04-25 13:21:15-- http://10.0.2.2:8000/Reducer.py
Connecting to 10.0.2.2:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 921 [text/x-python]
Saving to: "Reducer.py"

100%[=====] 921  --.-K/s  in 0s

2026-04-25 13:21:15 (85.3 MB/s) - "Reducer.py" saved [921/921]

[cloudera@quickstart Desktop]$ wget http://10.0.2.2:8000/Reducer.py
--2026-04-25 13:21:32-- http://10.0.2.2:8000/Reducer.py
Connecting to 10.0.2.2:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1589 (1.6K) [text/x-python]
Saving to: "Reducer.py"

100%[=====] 1,589  --.-K/s  in 0.003s
```



```
Applications Places System cloudera@quickstart:~/Desktop
File Edit View Search Terminal Help
2026-04-25 13:20:44 (8.37 MB/s) - "dataset.csv" saved [169490/169490]

[cloudera@quickstart Desktop]$ wget http://10.0.2.2:8000/Reducer.py
--2026-04-25 13:21:15-- http://10.0.2.2:8000/Reducer.py
Connecting to 10.0.2.2:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 921 [text/x-python]
Saving to: "Reducer.py"

100%[=====] 921  --.-K/s  in 0s

2026-04-25 13:21:15 (85.3 MB/s) - "Reducer.py" saved [921/921]

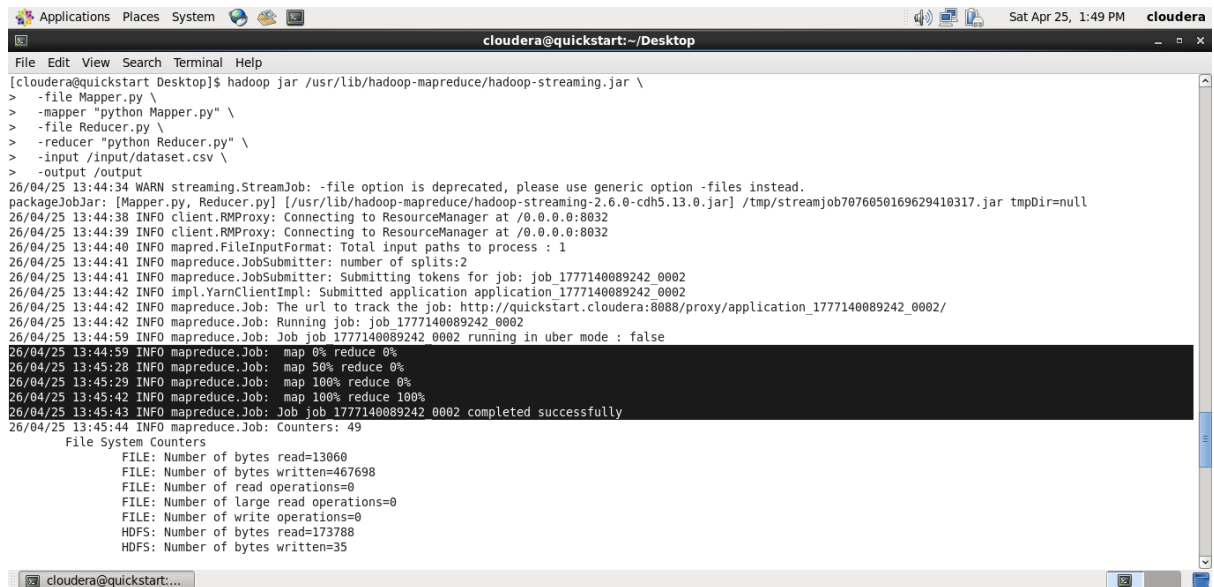
[cloudera@quickstart Desktop]$ wget http://10.0.2.2:8000/Reducer.py
--2026-04-25 13:21:32-- http://10.0.2.2:8000/Reducer.py
Connecting to 10.0.2.2:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1589 (1.6K) [text/x-python]
Saving to: "Reducer.py"

100%[=====] 1,589  --.-K/s  in 0.003s

2026-04-25 13:21:32 (460 KB/s) - "Reducer.py" saved [1589/1589]

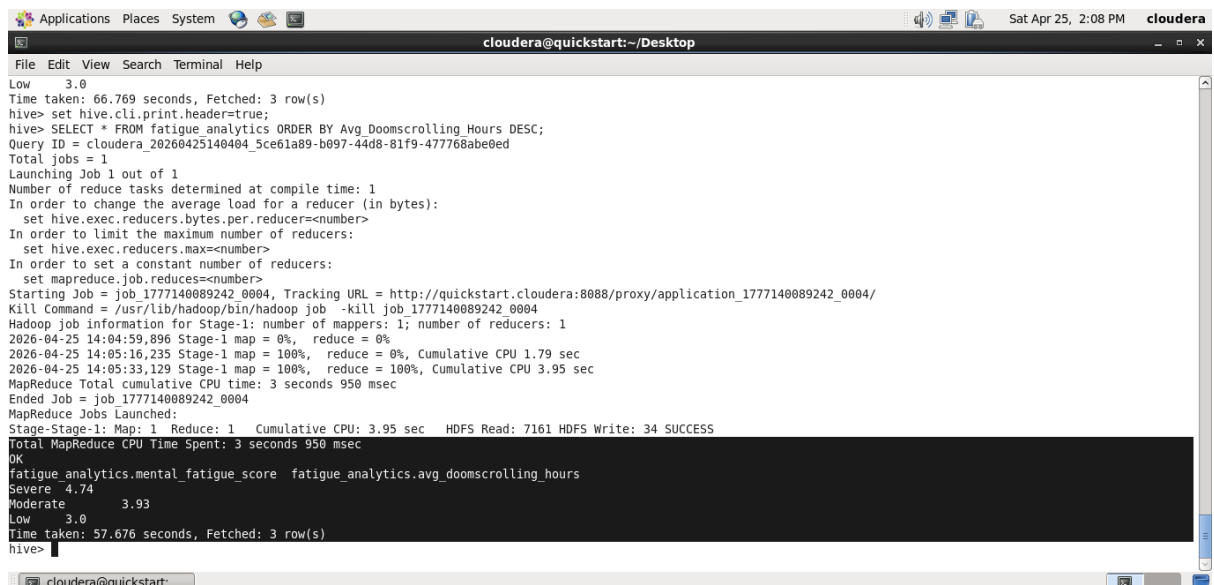
[cloudera@quickstart Desktop]$ ls
dataset.csv  Eclipse.desktop  Enterprise.desktop  Express.desktop  Kerberos.desktop  Mapper.py  Parcels.desktop  Reducer.py
[cloudera@quickstart Desktop]$ hdfs dfs -mkdir /input
[cloudera@quickstart Desktop]$ hdfs dfs -put dataset.csv /input/
[cloudera@quickstart Desktop]$ hdfs dfs -ls /input/
Found 1 items
-rw-r--r-- 1 cloudera supergroup 169490 2026-04-25 13:24 /input/dataset.csv
[cloudera@quickstart Desktop]$
```

- **Step 2: MapReduce Job Execution:** The distributed processing job was launched using `hadoop-streaming.jar`. The system distributed the workload across multiple map tasks, successfully filtering and aggregating the target metrics without data loss.



```
cloudera@quickstart:~/Desktop
File Edit View Search Terminal Help
[cloudera@quickstart Desktop]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
> -file Mapper.py \
> -mapper "python Mapper.py" \
> -file Reducer.py \
> -reducer "python Reducer.py" \
> -input /input/dataset.csv \
> -output /output
26/04/25 13:44:34 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [Mapper.py, Reducer.py] [/usr/lib/hadoop-mapreduce/hadoop-streaming-2.6.0-cdh5.13.0.jar] /tmp/streamjob7076050169629410317.jar tmpDir=null
26/04/25 13:44:38 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/25 13:44:39 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/25 13:44:40 INFO mapred.FileInputFormat: Total input paths to process : 1
26/04/25 13:44:41 INFO mapreduce.JobSubmitter: number of splits:2
26/04/25 13:44:41 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777140089242_0002
26/04/25 13:44:42 INFO impl.YarnClientImpl: Submitted application application_1777140089242_0002
26/04/25 13:44:42 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1777140089242_0002/
26/04/25 13:44:42 INFO mapreduce.Job: Running job: job_1777140089242_0002
26/04/25 13:44:59 INFO mapreduce.Job: Job job_1777140089242_0002 running in uber mode : false
26/04/25 13:44:59 INFO mapreduce.Job: map 0% reduce 0%
26/04/25 13:45:28 INFO mapreduce.Job: map 50% reduce 0%
26/04/25 13:45:29 INFO mapreduce.Job: map 100% reduce 0%
26/04/25 13:45:42 INFO mapreduce.Job: map 100% reduce 100%
26/04/25 13:45:43 INFO mapreduce.Job: Job job_1777140089242_0002 completed successfully
26/04/25 13:45:44 INFO mapreduce.Job: Counters: 49
File System Counters
FILE: Number of bytes read=13060
FILE: Number of bytes written=467698
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=173788
HDFS: Number of bytes written=35
cloudera@quickstart:~/Desktop
```

- **Step 3: Data Warehousing in Apache Hive:** The part-00000 output file was validated and loaded into a newly constructed Hive schema. A SELECT query utilizing an ORDER BY clause was executed to format and rank the final analytical results.



```
cloudera@quickstart:~/Desktop
File Edit View Search Terminal Help
Low 3.0
Time taken: 66.769 seconds, Fetched: 3 row(s)
hive> set hive.cli.print.header=true;
hive> SELECT * FROM fatigue_analytics ORDER BY Avg.Doomscrolling Hours DESC;
Query ID = cloudera_20260425140404_5ce61a89-b097-44d8-81f9-477768abe0ed
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1777140089242_0004, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1777140089242_0004/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1777140089242_0004
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-25 14:04:59,896 Stage-1 map = 0%, reduce = 0%
2026-04-25 14:05:16,235 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.79 sec
2026-04-25 14:05:33,129 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 3.95 sec
MapReduce Total cumulative CPU time: 3 seconds 950 msec
Ended Job = job_1777140089242_0004
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 3.95 sec HDFS Read: 7161 HDFS Write: 34 SUCCESS
Total MapReduce CPU Time Spent: 3 seconds 950 msec
OK
fatigue_analytics.mental_fatigue_score fatigue_analytics.avg_doomscrolling_hours
Severe 4.74
Moderate 3.93
Low 3.0
Time taken: 57.676 seconds, Fetched: 3 row(s)
hive>
cloudera@quickstart:~/Desktop
```

Core Logic Explanation

- **Mapper Logic:** The script reads the dataset line by line, bypassing the header. It applies a strict filtering condition, isolating only students who reported a stress level of 7 or higher. It emits a key-value pair consisting of Mental_Fatigue_Score and Doomscrolling_Hours.
- **Reducer Logic:** The script receives the sorted key-value pairs from the Mapper. It groups the data by the unique Mental_Fatigue_Score keys and calculates the precise average of Doomscrolling_Hours for each specific category, outputting the final analytical baseline.

4. Tools & Technologies Deployed

- **Ecosystem:** Cloudera Hadoop Distribution.
- **Storage:** Hadoop Distributed File System (HDFS).
- **Processing:** MapReduce (Hadoop Streaming with Python).
- **Analytics:** Apache Hive (HiveQL).

Overview of AI Integration

- Generative AI was strategically deployed as a pair-programming and debugging assistant throughout the development lifecycle of this Big Data pipeline.
- Instead of relying on pre-cleaned, generic datasets, AI was utilized to architect a custom, highly specific schema that perfectly matched the project's analytical hypothesis regarding digital productivity.

Specific Areas of AI Implementation

- **1. Synthetic Data Engineering:**

AI was prompted to write a Python generation script that created a flawless 5,000-row CSV file. The prompt strictly defined the data types (Float, Integer, String) and the logical correlation rules (e.g., higher probability of 'Severe' mental fatigue when sleep is low and screen time is high).

- **2. Algorithmic Code Generation:**

AI assisted in drafting the foundational Mapper.py and Reducer.py logic, specifically optimizing the code to read from sys.stdin to ensure seamless compatibility with the Hadoop Streaming utility.

- **3. Iterative Debugging & Syntax Resolution:**

During the initial MapReduce execution, the Hadoop container threw a critical subprocess failed with code 1 exception. AI was utilized to analyze the traceback.

It successfully identified an environment mismatch: the Cloudera CentOS environment defaulted to Python 2.7, which rejected the Python 3 f-string formatting generated in the

Reducer. The AI provided the immediate solution to refactor the print statements using the legacy `.format()` method, resolving the cluster failure instantly.

- **4. Query Formulation:**

AI was used to ensure the Hive CREATE TABLE statement perfectly mapped its data types and tab-separated delimiters to the precise output format generated by the Reducer.

Benefits of AI-Assisted Engineering

- **Accelerated Development Pipeline:** By automating the dataset creation and initial boilerplate scripting, the development focus was successfully shifted from manual data entry to core systems architecture and logic validation.
- **Advanced Error Reduction:** The AI's ability to instantly translate obscure Java/Hadoop wrapper errors into actionable Python syntax fixes saved hours of manual documentation searching.
- **Enhanced Logical Clarity:** Collaborating with AI required writing highly specific, constraint-based prompts, which fundamentally reinforced a deeper understanding of how data flows through a MapReduce pipeline.